

光辰智能 Agent Harness 架构设计答案

整理日期：2026-10-05。以下是本题的设计判断与可验证方案，涉及扩展能力时会说明与当前代码的区别。没有用虚构项目经历或性能数字替代论证。

选题与总体判断

模块	选择	为什么选这道题
一 Context 与性能	第 2 题，200 轮上下文压缩	直接讨论已有原型中的压缩、协议配对和信息损失
二 Memory	第 1 题，半个月前的问题召回	说明检索时机、记忆有效性和窗口边界
三 Task	第 1 题，长程任务目标漂移	比较几种方案，而非把计划清单等同于完成证据
四 Tool 与 Session	第 2 题，busy 时的新输入与异步结果	从单写者出发处理取消、重复事件和迟到结果
五 架构比较	第 2 题，OpenHands 状态机优缺点与改进	结合本地同步循环解释何时需要事件化架构

我的总体选择是先实现足够小、能解释和测试的同步原型，再围绕明确的失败场景扩展。更多历史或更复杂的框架不自动提高答案质量；应看是否保留了本轮决策所需的证据，是否约束了副作用，以及失败后是否能恢复。

模块一 第 2 题 200 轮上下文压缩

我的判断

问题不是把 200 轮都送进去，而是用户继续追问时，目标、约束、已确认事实和未完成工作仍然可用。全量历史应可回查，模型上下文则是有预算的投影。例如第一轮约定“预算 3000 元”，第一百轮修正为“2500 元”，第二百轮问“之前那个方案还能做吗”，只保留最后几条消息会忘记预算，反复摘要又可能把两个版本混合。

方案

保存原始消息与工具事件，每次模型调用前构造上下文：稳定 system 规则、带来源的任务与历史资料、最近完整用户轮次。近期轮次里的 assistant 工具调用和全部对应 tool 结果一起保留或移除，不能直接按最后 N 条消息切片。大结果优先分页读取或引用原文，避免工具输出反复占用上下文。

接近预算时把旧轮次整理为结构化摘要，字段包括目标、约束、已确认事实、决定、未决事项、时间及来源事件 ID。模型生成摘要后检查日期、金额、编号和工具结果等关键字段。修正记录使旧事实失效，但保留变更来源。周期性从原始事件重建摘要，减少递归摘要的累计失真。

召回不是一次性预处理，而是在每次回答前根据当前问题与近期话题检索旧原文，补充少量相关证据。指代不清或证据不足时澄清。生产预算使用对应模型 tokenizer，并预留输出及 observation 空间；当前完整任务仍超限则停在安全检查点或拆分任务。

取舍与验证

滑动窗口实现简单，却容易丢早期约束；纯摘要便宜，却难恢复被遗漏细节；窗口、摘要和按需召回组合更稳，但增加检索和摘要维护成本。我会先验证早期约束丢失是否是主要错误，再决定是否值得加入模型摘要。评价指标是关键事实正确率、未决任务保留率、工具协议配对、成本及延迟，而非压缩率越大越好。

评测使用同一批长对话，覆盖早期预算、用户纠错、代词追问、长工具链和双窗口隔离，比较完整上下文与压缩上下文的答案。当前代码已有 200 轮协议配对、关键词命中及长答案末尾事实的离线测试；采用字符预算和截取式摘录，没有 tokenizer、语义检索或结构化事实版本。因此这些测试证明有限场景正确，不代表任意 200 轮都能恢复。

模块二 第 1 题 半个月前的问题召回

我的判断

先区分“上次我选择了什么”和“现在应该选择什么”。前者回答历史决定，后者还要检查新约束和事实是否变化。半个月不是失效的充分条件，资料类别与用户修正更关键。比如旧答案采用套餐 A，之后用户明确改为 B，再次提问时不能仅因为 A 的文本更相似就沿用 A。

方案

记忆分为 session 中的临时任务与用户明确允许跨窗口使用的稳定事实。记录保存用户、作用域、来源、时间、有效期和修正关系。查询先过滤权限与作用域，再做检索和排序，避免其他用户或窗口的临时数据进入上下文。

每次回答前，用当前问题与最近话题构造查询。关键词处理项目名、日期和编号；语义检索处理同义表达；重排结合相关性、时间及来源可信度。选取少量可追溯片段，标明时间与原文编号，放入上下文的数据区。历史指令不能获得新的 system 权限。

命中后检查有效状态与后续修正。如果问过去的决定，应说明当时的选择；如果问当前建议，旧记录只作背景，必要时更新外部资料。没有命中只能说明记录不足，不能断言用户从未提问。纠错新增修正关系并更新索引，删除同步处理原文、事实状态和检索索引。

取舍与验证

关键词检索便宜、易解释，精确名称较好，但“价格上限”和“预算”可能匹配失败。向量检索能处理同义表达，也会召回看似相似但不适用的旧方案。因此我不会把向量相似度当事实有效性的判断；应先限定作用域，再结合版本与来源验证。

评测覆盖精确复问、同义复问、过期信息、后续纠错、跨话题和跨用户隔离。检索指标与最终回答正确率分开统计：召回 A 和 B 不等于模型会选对 B。当前代码每次模型请求前，在同 session 的旧 user/assistant 文本中取最多两条关键词命中，带轮次编号注入 user 数据消息。尚无跨窗口偏好、语义召回、鉴权或过期管理。

模块三 第 1 题 长程任务目标漂移

我的判断

目标需要可恢复的状态与完成证据，不能只靠模型记住最初提示。例如任务是“实现购物 Agent 并给出测试结果”，中途加入搜索工具后，模型可能继续优化搜索，忽略最终测试与提交。完成了很多步骤不等于完成原始目标。

备选方案

方案一是每轮加入精简目标与完成条件，实现成本最低，适合短任务；但重复提示会消耗预算，目标表述错误也会持续强化。方案二是分阶段计划与持久化检查点，保存当前阶段、完成证据、下一步、阻塞项和剩余预算，适合恢复与重规划；代价是维护状态，且计划需要随新证据更新。方案三是独立评估器或规则验收，能发现偏航，但额外模型评估有费用和误判风险。

我会组合目标卡、检查点和确定性验收，把额外评估器放在少数高价值节点。例如代码任务要求测试通过、产物存在、说明与实现一致；不能只接受模型写出“完成”。出现连续无进展、重复失败或预算超限时先检查缺少的信息和外部阻塞，再重规划，避免无限追加步骤。

状态与恢复

task state 保存目标版本、验收条件、当前计划、证据引用及预算。用户改目标时新增版本，在下一次副作用前检查版本，旧任务证据只有仍适用的部分能复用。恢复先读取完成记录，再判断继续还是重新执行。外部写入使用业务幂等键；若无法确认结果，标记未知并查询，不直接重复提交。

我只保存简短决策依据、行动与结果，不依赖隐藏推理链作为恢复协议。一个目标可以有不同计划，重规划应改变路径而非悄悄改变目标。

取舍与验证

测试注入中途修改目标、工具多次失败、重启、重复执行以及模型口头宣告完成但产物不合格。比较目标保持、恢复后重复副作用和验收误判情况。当前代码有步数上限、todo、最初请求线索与 session 历史，但无独立 task 表、阶段验收器和断点恢复。本题中的 task state 属于扩展方案。

模块四 第 2 题 busy 时的新输入与异步结果

我的判断

busy 是正在执行的状态，不能作为丢弃已确认输入的理由。核心约束是每个 session 同时只有一个状态写入者。典型竞态是模型已决定付款，用户发送“取消”，工具完成事件又刚好到达；消息顺序和副作用边界必须明确。

方案

接入层校验身份与作用域，将用户输入和异步完成写成持久化 inbox 事件，包含 event_id、会话内序号、task_id、目标版本及调用关联。落库后确认已接收；普通输入排队，取消与目标修正作为控制事件，在下次副作用前的安全边界处理。队列设置积压上限与背压，避免无限接收。

每个 session 由串行 worker/reducer 更新状态。模型使用请求发送时的快照，新输入不能修改正在进行的请求；响应返回后检查取消状态与目标版本，再执行工具。取消后的旧响应可以保留审计，不继续过时写操作。已经发生的外部副作用应如实反馈，取消不等于自动撤销。

异步工具先返回 pending 与 job_id，最终完成形成独立事件；不用长期占着模型请求轮询。按 job_id 和 task_id 回填，重复 event_id 不重复消费，迟到结果不重新激活已取消任务。消费标记与状态更新同事务提交，通知通过 outbox 独立重试，通知失败不能重新执行已经成功的工具。

取舍与验证

单写者减少状态竞争，却可能造成排队；异步 executor 可并发执行不同 session 的慢任务。多进程需数据库版本或租约，并拒绝过期 worker 的写入。事件去重解决重复消费，外部业务幂等解决重复副作用，这两个机制不能互相替代。

测试覆盖用户消息与工具结果同时到达、重复事件、取消后成功、落库后崩溃重放及通知失败。当前原型对同一进程、同一规范化数据库路径的活动 session 返回 busy，并提示重试；没有 inbox、异步工具或跨进程租约。此边界已在 README 与测试中明确。

模块五 第 2 题 OpenHands 状态机优劣与改进

比较范围

我主要讨论旧 AgentController/EventStream 路线，并参考后续 SDK 的模块边界。官方 2025-09-04 的 v1 迁移说明讨论了旧 pub/sub EventStream 的顺序、线程与异步复杂性，并提出同步 conversation 执行方向。当前 SDK 文档将 Agent、Conversation、Tools、Workspace 等职责分开，所以不能笼统认为所有版本都缺少边界。以下改进是我的设计判断。

优点与问题

显式状态机把生命周期及允许的动作表达出来，Action/Observation 关联调用与结果，利于审计、检查非法迁移及恢复。问题通常发生在多个回调共同推进 state：重复结果、迟到结果、取消与预算结束同时发生，谁有权改变当前任务？如果一个控制器还承包模型、工具、用户输入、子任务和重试，耦合会进一步增加。

增加更多状态枚举不能独立解决这些竞态。需要明确事件顺序、状态所有权、关联信息与副作用执行边界。

我的改进

保留显式迁移，将每 session 的状态变化收敛到纯 reducer：旧状态加事件，产生新状态和 effects；模型、工具、存储与通知各有独立模块。executor 执行 effects 后把结果写回 inbox，reducer 不直接发网络请求，便于用事件序列测试与重放。

事件带任务版本和调用关联，检查点记录待执行与已完成 effects。状态更新与消费记录原子提交，外部动作有业务幂等键。重放用于重建状态，不无条件重新执行外部操作；取消任务的迟到结果仅记录审计。若外部服务无法提供幂等或查询，就需要显式处理结果未知的状态。

为什么本次没有直接上完整事件系统

事件化增加版本、恢复、积压、幂等与运维成本。本次本地工具执行快、单进程 CLI 也容易解释，所以同步循环是合理起点。只有慢工具、多 worker、长程恢复和可靠通知成为实际要求时，再增加 durable inbox 和单写者协调。我会用重复完成事件、取消竞态和重放测试验证改进，而非以代码层数更少作为“更优雅”的证据。